DOCUMENTAÇÃO TÉCNICA · LINGUAGEM C

PSE em Ação

Como o código está organizado e como cada parte funciona

Este guia apresenta o arquivo `pse.c` em linguagem direta. Ele complementa o `README.md`, que explica como usar o programa.

Projeto de Engenharia de Software · 2º semestre

Neste documento

Uma visão organizada das partes do programa, das regras e das funções.

1. Visão geral
2. Constantes
3. Como os dados são organizados
4. Casos de uso e regras
5. Funções
6. Fluxos principais
7. Tratamento de erros
8. Decisões de implementação

Resumo do projeto

O programa cadastra e acompanha ações do Programa Saúde na Escola. Os registros ficam em um vetor durante a execução e são guardados no arquivo `acoes.txt` para serem recuperados na próxima abertura.

1. Visão geral

Item	Descrição
Linguagem	C, usando o padrão C99 na compilação.
Arquivo do código	pse.c, com cerca de 480 linhas.
Bibliotecas	Apenas stdio.h e stdlib.h.
Armazenamento	As ações ficam em uma lista na memória e são salvas no arquivo acoes.txt.
Compilação	gcc -Wall -Wextra -std=c99 -o pse.exe pse.c.

O comando de compilação

No comando abaixo, `-o pse.exe` dá esse nome ao programa compilado. Para executar no PowerShell, use `.\pse.exe`.

```
gcc -Wall -Wextra -std=c99 -o pse.exe pse.c
.\pse.exe
```

O código está organizado em cinco partes: definições, funções de texto, funções auxiliares, leitura e gravação de arquivo e funções do menu.

2. Constantes e dados

Uma constante é um valor com nome, usado pelo programa para manter regras em um único lugar.

Constante	Valor	Uso
MAX_ACOES	100	Máximo de ações armazenadas.
TAM_TEXTO	60	Até 59 caracteres de texto mais o final da string.
ARQUIVO_DADOS	“acoes.txt”	Nome do arquivo que guarda os registros.
PLANEJADA	1	Código da situação Planejada.
REALIZADA	2	Código da situação Realizada.
CANCELADA	3	Código da situação Cancelada.

Estrutura Acao

A palavra `struct` agrupa informações relacionadas. Aqui, cada ação tem seus próprios campos:

```
typedef struct {
    int codigo;
    char escola[TAM_TEXTO];
    char tema[TAM_TEXTO];
    char data[12];
    char publico[TAM_TEXTO];
    char responsavel[TAM_TEXTO];
    int qtdPrevista;
    int qtdEfetiva;
    int situacao;
} Acao;
```

Campo	Regra
codigo	Número único, de 1 a 999999.
escola, tema, publico e responsavel	Textos obrigatórios; não podem conter ponto e vírgula.
data	Formato dd/mm/aaaa; data real entre 2000 e 2100.
qtdPrevista	De 1 a 100000.
qtdEfetiva	De 0 a 100000; vale -1 antes do registro de participantes.
situacao	PLANEJADA, REALIZADA ou CANCELADA.

O campo `data` tem 12 posições: 10 para a data, uma para `\0` e uma de folga. Isso ajuda a detectar uma data digitada com um caractere a mais.

Lista de ações

```
Acao acoes[MAX_ACOES];
int totalAcoes = 0;
```

O vetor `acoes` guarda até 100 ações. As posições ocupadas vão de 0 até `totalAcoes - 1`. Uma nova ação entra na próxima posição livre.

3. Casos de uso e regras

Caso de uso é o nome dado a uma tarefa que o usuário pode realizar no sistema.

Tarefa	Função no código
Cadastrar ação	cadastro()
Listar ações	lista()
Pesquisar ações	pesquisa()
Atualizar situação	atualSituacao()
Registrar participantes	regParticip()
Gerar resumo	geraResumo()
Validar informações	lerTexto(), lerInteiro(), dataValida() e verificação de código repetido em cadastro()

Regras aplicadas

Regra	Onde é verificada
O código não pode se repetir.	cadastro() procura usando buscarPorCodigo().
Campos obrigatórios não podem ficar vazios.	lerTexto() pede o texto novamente.
Quantidades precisam respeitar o intervalo permitido.	lerInteiro() confere os valores mínimos e máximos.
As opções do menu e das situações precisam ser válidas.	lerInteiro() limita os valores aceitos.
A situação é planejada, realizada ou cancelada.	atualSituacao() aceita opções de 1 a 3.
Só ações realizadas podem receber participantes.	regParticip() confere a situação antes do registro.
A pesquisa aceita código, escola ou tema.	pesquisa() aceita critérios de 1 a 3.
O limite é de 100 ações.	cadastro() confere MAX_ACOES.

4. Funções do programa

4.1 Funções de texto

Como o projeto usa apenas `stdio.h` e `stdlib.h`, algumas operações com texto foram escritas no próprio programa.

Função	O que faz
tamanhoTexto(texto)	Conta os caracteres até encontrar \0.
temCaractere(texto, c)	Confere se um caractere aparece no texto; é usada para recusar ;.
minusculo(c)	Transforma uma letra maiúscula em minúscula.
textosIguais(a, b)	Compara dois textos sem diferenciar maiúsculas de minúsculas.

Função	O que faz
contemTexto(texto, busca)	Confere se um trecho aparece dentro de um texto.

Por exemplo, a pesquisa por tema pode encontrar “Buc” dentro de “Saude Bucal”. A função testa os pontos do texto até encontrar o trecho.

4.2 Leitura e validação

O programa usa `fgets` para ler uma linha inteira, inclusive textos com espaços. Com `scanf("%s")`, “EM Joao Paulo” seria lido apenas até o primeiro espaço. A leitura numérica com `scanf("%d")` também exige cuidado quando o usuário digita uma letra.

Função	O que faz
lerLinha(destino, tamanho)	Lê uma linha, remove o ENTER e descarta o excesso de caracteres. Se a entrada acabar (EOF), encerra o programa.
lerTexto(pergunta, destino, tamanho)	Lê um texto obrigatório. Pede novamente se ficar vazio ou contiver ;.
lerInteiro(pergunta, minimo, maximo)	Lê um número e só aceita valores dentro do intervalo indicado.
dataValida(data)	Confere o formato e verifica se a data existe.

`lerInteiro()` usa `strtol` para transformar o texto digitado em número. Se sobrarem letras depois do número, a entrada é recusada. A função também avisa quando o número está fora do intervalo.

`dataValida()` verifica se a data tem o formato `dd/mm/aaaa`, se as barras estão nas posições certas e se o dia existe naquele mês. Também considera fevereiro em anos bissextos.

4.3 Busca e exibição

Função	O que faz
buscarPorCodigo(codigo)	Procura o código na lista. Retorna a posição da ação ou -1 se não encontrar.
nomeSituacao(situacao)	Transforma 1, 2 ou 3 em Planejada, Realizada ou Cancelada.
mostrarAcao(acao)	Mostra os dados de uma ação. Indica quando os participantes ainda não foram registrados.
pausar()	Espera o usuário apertar ENTER para que o resultado continue visível.

A busca por código verifica as ações uma por uma. Como a lista tem no máximo 100 itens, essa busca é suficiente para o tamanho do projeto.

4.4 Arquivo de dados

Função	O que faz
salvarDados()	Grava todas as ações em acoes.txt, uma por linha, com campos separados por ;. É chamada após alterações.
carregarDados()	Lê acoes.txt quando o programa inicia e coloca as ações no vetor.

Se o arquivo ainda não existir, o programa começa vazio. Linhas incompletas ou com formato incorreto são ignoradas. A leitura para ao chegar ao limite de 100 ações.

Exemplo de uma linha do arquivo:

```
10;EM Joao Paulo;Saude Bucal;15/03/2026;Alunos do 5 ano;Enf. Maria;40;35;2
```

4.5 Funções do menu

Função	Resumo
menu(op)	Mostra o menu e a quantidade de ações cadastradas.
cadastro()	Confere o limite e o código, pede os dados, inicia a ação como planejada e salva.
lista()	Mostra as ações em uma tabela ou avisa se não houver registros.
pesquisa(&resposta)	Pesquisa por código, escola ou tema e informa se houve resultados.
atualSituacao()	Procura a ação e altera sua situação. Se ela deixar de ser realizada, remove o registro de participantes.
regParticip()	Permite informar participantes apenas quando a ação está realizada.
geraResumo()	Conta ações e participantes e organiza os totais por escola.
main()	Carrega os dados, mostra o menu e chama a função escolhida.

Em `pesquisa(&resposta)`, o símbolo & envia para a função o endereço da variável. Assim, a função pode devolver a resposta para `main`, que repete a pesquisa enquanto a resposta for “S”.

5. Fluxos principais

Programa principal

Início → carregar arquivo → mostrar menu → ler opção → executar função escolhida → pausar → voltar ao menu. A opção 7 mostra a despedida e encerra o programa. A pesquisa pode se repetir quando o usuário responde “S”.

Cadastro de ação

Passo	O que acontece
1	Confere se o limite de 100 ações foi atingido.
2	Pede o código. O código 0 cancela; um código repetido precisa ser trocado.
3	Pede escola, tema, data, público, responsável e quantidade prevista.
4	Se a data não for válida, pede outra.
5	Define a situação como Planejada e qtdEfetiva como -1.
6	Guarda a ação no vetor, salva o arquivo e confirma.

Registro de participantes

Passo	O que acontece
1	Pede o código da ação.
2	Se não encontrar a ação, informa e volta ao menu.
3	Se a ação não estiver realizada, não permite o registro.
4	Se estiver realizada, pede a quantidade efetiva, salva e confirma.

6. Tratamento de erros

Situação	Onde é tratada	Resposta do programa
Letra no lugar de número	lerInteiro()	“Digite apenas numeros inteiros.”
Número fora do intervalo	lerInteiro()	“Valor invalido. Digite entre X e Y.”
Número negativo	lerInteiro()	“O valor nao pode ser negativo.”
Campo vazio	lerTexto()	“Campo obrigatorio. Preencha, por favor.”
Ponto e vírgula em texto	lerTexto()	“O caractere ; nao e permitido.”
Texto maior que o campo	lerLinha()	Descarta o excesso sem travar.
Data inválida	cadastro() e dataValida()	Pede uma data válida no formato dd/mm/aaaa.
Código repetido	cadastro()	Pede outro código.
Código não encontrado	pesquisa(), atualSituacao(), regParticip()	Informa que não encontrou a ação.

Situação	Onde é tratada	Resposta do programa
Limite atingido	cadastro()	Informa o limite de 100 ações.
Participantes em ação não realizada	regParticip()	Informa que a ação precisa estar realizada.
Lista ou resumo sem dados	lista(), geraResumo()	Informa que não há ações ou dados.
Falha ao salvar arquivo	salvarDados()	Mostra um aviso.
Arquivo não existe	carregarDados()	O programa começa vazio.
Linha do arquivo incorreta	carregarDados()	Ignora a linha.

7. Decisões de implementação

- Vetor com limite de 100 ações: deixa o programa mais simples e permite avisar quando o limite for atingido.
- Salvar o arquivo inteiro depois de uma alteração: com até 100 ações, é rápido e mantém o arquivo atualizado.
- Separar os campos por ponto e vírgula: facilita conferir os dados em um editor de texto ou planilha.
- Mensagens sem acentos: evita problemas de exibição no console do Windows.
- system("cls"): limpa a tela no Windows. Em Linux ou macOS, normalmente é usado system("clear").

Fim da documentação do código. Para compilar e usar o programa, consulte também o README.md.